

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AI & ML
LABORATORY OBSERVATION RECORD

BANK ACCOUNT SYSTEM - OBJECT ORIENTED PROGRAMMING IN JAVA

1. TITLE OF THE EXPERIMENT

Implementation of a Bank Account System using Object-Oriented Programming concepts such as classes, objects, constructors, inheritance, method overriding, deposits, withdrawals, interest calculation and fund transfer in Java.

2. AIM

To design and implement a Java Bank Account System using Object-Oriented Programming concepts by creating a base Account class and derived SavingsAccount and CurrentAccount classes, and to demonstrate object creation, constructors, inheritance, method overriding, deposit, withdrawal, interest calculation and fund transfer operations.

3. ALGORITHM

Step 1: Start the program.

Step 2: Define the Account class with account number, account holder name, balance and account type.

Step 3: Define a constructor in Account to initialize the account details.

Step 4: Implement deposit() to add a valid amount to the account balance.

Step 5: Implement withdraw() to withdraw money only when sufficient balance is available.

Step 6: Implement transfer() to transfer money from one Account object to another when sufficient balance is available.

Step 7: Implement displayAccountDetails() to display the account information.

Step 8: Define SavingsAccount as a subclass of Account and add interestRate.

Step 9: Implement calculateInterest() in SavingsAccount and add the calculated interest to the balance.

Step 10: Define CurrentAccount as a subclass of Account and add overdraftLimit.

Step 11: Override withdraw() in CurrentAccount so withdrawal is allowed up to balance plus the permitted overdraft limit.

Step 12: Create SavingsAccount and CurrentAccount objects in main().

Step 13: Display the initial account details.

Step 14: Perform deposits, withdrawals and interest calculation.

Step 15: Transfer money from the savings account to the current account.

Step 16: Display the final account details and balances.

Step 17: Stop the program.

4. EXECUTION STEPS

Step 1: Compile the Java source file using the package name task and class name BankSystem.

Step 2: Run the BankSystem program.

Step 3: Observe the INITIAL ACCOUNT DETAILS section and verify the SavingsAccount and CurrentAccount values.

Step 4: Observe the SAVINGS ACCOUNT OPERATIONS section and verify deposit, withdrawal and interest calculation.

Step 5: Observe the CURRENT ACCOUNT OPERATIONS section and verify deposit and withdrawal using the overdraft limit.

Step 6: Observe the FUND TRANSFER section and verify the transfer from the savings account to the current account.

Step 7: Observe the FINAL ACCOUNT DETAILS section and verify the final balances.

Step 8: Compare the displayed output with the expected output.

5. SOURCE CODE

```
package task;

//Bank Account System using OOP concepts

//Base class: common details and operations of every account

class Account {

    int accountNumber;
    String accountHolderName;
    double balance;
    String accountType;

    // Constructor
    Account(int accountNumber, String accountHolderName,
            double balance, String accountType) {

        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
        this.accountType = accountType;
    }

    // Deposit money
    void deposit(double amount) {

        if (amount > 0) {
            balance = balance + amount;
            System.out.println("Deposited: " + amount);
        } else {
            System.out.println("Invalid deposit amount");
        }
    }

    // Withdraw money
    void withdraw(double amount) {

        if (amount > 0 && amount <= balance) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient balance");
        }
    }

    // Transfer money
    void transfer(Account receiver, double amount) {

        if (amount > 0 && amount <= balance) {
            balance = balance - amount;
            receiver.balance = receiver.balance + amount;
            System.out.println("Transferred: " + amount);
        } else {
            System.out.println("Transfer failed");
        }
    }

    // Display account details
    void displayAccountDetails() {

        System.out.println("Account Number : " + accountNumber);
        System.out.println("Account Holder : " + accountHolderName);
        System.out.println("Account Type : " + accountType);
        System.out.println("Balance : " + balance);
    }
}

// SavingsAccount inherits Account

class SavingsAccount extends Account {

    double interestRate;

    // Constructor
    SavingsAccount(int accountNumber, String accountHolderName,
            double balance, double interestRate) {

        super(accountNumber, accountHolderName, balance, "Savings");
        this.interestRate = interestRate;
    }
}
```

```

    }

    // Calculate interest
    void calculateInterest() {

        double interest = balance * interestRate / 100;
        balance = balance + interest;
        System.out.println("Interest Added: " + interest);
    }
}

// CurrentAccount inherits Account
class CurrentAccount extends Account {

    double overdraftLimit;

    // Constructor
    CurrentAccount(int accountNumber, String accountHolderName,
                    double balance, double overdraftLimit) {

        super(accountNumber, accountHolderName, balance, "Current");
        this.overdraftLimit = overdraftLimit;
    }

    // Method overriding
    @Override
    void withdraw(double amount) {

        if (amount > 0 && amount <= balance + overdraftLimit) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Withdrawal exceeds overdraft limit");
        }
    }
}

//Main class: creates objects and performs transactions
public class BankSystem {

    public static void main(String[] args) {

        // Creating SavingsAccount object
        SavingsAccount savings =
            new SavingsAccount(101, "Raki", 10000, 5);

        // Creating CurrentAccount object
        CurrentAccount current =
            new CurrentAccount(102, "Priya", 5000, 3000);

        System.out.println("===== INITIAL ACCOUNT DETAILS =====");

        System.out.println("\nSavings Account");
        savings.displayAccountDetails();

        System.out.println("\nCurrent Account");
        current.displayAccountDetails();

        // Savings account operations
        System.out.println("\n===== SAVINGS ACCOUNT OPERATIONS =====");

        savings.deposit(2000);
        savings.withdraw(1000);
        savings.calculateInterest();

        // Current account operations
        System.out.println("\n===== CURRENT ACCOUNT OPERATIONS =====");

        current.deposit(3000);
        current.withdraw(9000);

        // Transfer money from savings to current
        System.out.println("\n===== FUND TRANSFER =====");

        savings.transfer(current, 2000);

        // Final details
        System.out.println("\n===== FINAL ACCOUNT DETAILS =====");
        System.out.println("\nSavings Account");
    }
}

```

```
savings.displayAccountDetails();

System.out.println("\nCurrent Account");
current.displayAccountDetails();
}
```

6. OUTPUT / EXECUTION

===== INITIAL ACCOUNT DETAILS =====

Savings Account
Account Number : 101
Account Holder : Raki
Account Type : Savings
Balance : 10000.0

Current Account
Account Number : 102
Account Holder : Priya
Account Type : Current
Balance : 5000.0

===== SAVINGS ACCOUNT OPERATIONS =====

Deposited: 2000.0
Withdrawn: 1000.0
Interest Added: 550.0

===== CURRENT ACCOUNT OPERATIONS =====

Deposited: 3000.0
Withdrawn: 9000.0

===== FUND TRANSFER =====

Transferred: 2000.0

===== FINAL ACCOUNT DETAILS =====

Savings Account
Account Number : 101
Account Holder : Raki
Account Type : Savings
Balance : 9550.0

Current Account
Account Number : 102
Account Holder : Priya
Account Type : Current
Balance : 1000.0

7. OUTPUT EXPLANATION

- Initial Details: The program creates one SavingsAccount and one CurrentAccount and displays their initial balances.
- Savings Account: ₹2,000 is deposited, ₹1,000 is withdrawn and 5% interest on ₹11,000 adds ₹550, giving ₹11,550 before the transfer.
- Current Account: ₹3,000 is deposited. The withdrawal of ₹9,000 is permitted because the account has a ₹3,000 overdraft limit.
- Fund Transfer: ₹2,000 is transferred from the savings account to the current account.
- Final Balances: SavingsAccount = ₹9,550.0 and CurrentAccount = ₹1,000.0.

8. RESULT

The Bank Account System was successfully implemented and executed. The program demonstrates classes, objects, constructors, inheritance, method overriding, deposit, withdrawal, interest calculation, overdraft handling and fund transfer operations in Java.

9. LEARNING OUTCOME

Understood how a base class can provide common account operations while SavingsAccount and CurrentAccount extend it with specialized features. Learned the use of constructors, inheritance, method overriding, objects and transaction methods to build a simple reusable banking application.